

Predicting Traffic Accident Severity

A worked case study on Addis Ababa's real accident records, motivated by
Vietnam's own traffic-safety problem

Your Name

2026-09-29

Table of contents

1	Introduction	2
2	Data	2
2.1	The central data-understanding decision: excluding casualty outcomes	3
2.2	Real data-quality issues, found by checking rather than assuming	4
3	Digging deeper: what actually correlates with severity?	4
3.1	Why Cramér's V, not a plain correlation matrix	4
3.2	The clearest example of why this matters: vehicle count	5
3.3	Light conditions, driver age, and time of day	6
4	Feature engineering	9
5	Choosing the modeling target: binary Severe, not raw 3-class	10
6	Methodology	11
6.1	Why macro-F1, made concrete	11
6.2	Class-imbalance handling, compared honestly	12
6.3	Model comparison	13
7	Choosing a decision threshold	14
8	Model interpretability	16
8.1	Checking SHAP against the EDA — and catching my own mistake	17
8.2	What this actually says, direction included	18
9	What this means in practice: policymakers and drivers	18

10 Discussion: what would a congestion-forecasting system need?	20
11 Conclusion	21

1 Introduction

I started this project because traffic accidents and congestion are such a visible, everyday problem in Vietnam's cities, and I wanted to try applying what I've learned about classification and imbalanced data to a real road-safety question. The obvious first step — finding a Vietnamese accident dataset — turned out to be a dead end: I checked, and Hanoi's accident records are paper-based police forms, not open data anywhere I could find.

So instead I picked a real, well-documented dataset chosen for its **contextual** similarity rather than its origin: accident records from Addis Ababa sub-city police (2017-2020). It's a developing-country, mixed-vehicle urban traffic environment — motorcycles, pedestrians, buses, and private cars sharing the same roads, with less-regulated driving conditions — which felt a lot closer to Vietnam's own traffic reality than the larger, better-known Western datasets I also considered (US-Accidents, UK STATS19), which are purely car-centric highway/suburban data. I want to be clear about what this claim is and isn't: I'm not saying Ethiopian and Vietnamese traffic are the same, only that the *kind* of problem — mixed-vehicle, developing urban road network — is far more comparable than a US freeway dataset would be.

My task: predict accident **severity** from conditions known *before* or *at* the moment of a crash — never from what happened to the people involved afterward. The raw data records severity as 3 classes (Slight / Serious / Fatal injury), but as I'll show below, I ended up deliberately collapsing that to a binary **Severe vs. Not Severe** target instead, after measuring — not assuming — that the 3-class version couldn't reliably detect the outcome that matters most.

I also didn't want this to be a project that jumps straight into modeling and treats the data itself as a formality. A good chunk of this report is a genuinely deep look at the data first — which raw factors actually associate with severity, using a method built for categorical data instead of a plain correlation matrix that would have missed the most important pattern in the dataset — and, once a model exists, a deliberate effort to translate what it finds into concrete recommendations for two audiences who'd actually care about the answer: people who set road-safety policy, and people who just drive on these roads every day.

2 Data

12,316 real accident records, 32 raw columns.

```

from traffic_accident_severity import config, data

df = data.load_accidents()
print(f"{len(df)} rows, {df.shape[1]} columns")
df[config.TARGET_COL].value_counts(normalize=True)

```

12316 rows, 32 columns

```

Accident_severity
Slight Injury      0.845648
Serious Injury     0.141523
Fatal injury       0.012829
Name: proportion, dtype: float64

```

2.1 The central data-understanding decision: excluding casualty outcomes

Looking through the raw columns, I noticed they split into two fundamentally different kinds:

- **Pre-accident risk factors** (driver, vehicle, road, weather, time, collision dynamics) — knowable before/at the crash, the only kind of feature a real prevention or policy system could act on.
- **Post-accident casualty-outcome columns** (Casualty_class, Casualty_severity, Sex_of_casualty, Age_band_of_casualty, Work_of_casualty, Fitness_of_casualty, Number_of_casualties) — recorded *after* the crash, describing who got hurt and how.

This second group is a data-leakage trap I nearly walked into: using these as features would be circular, since a system that needs to already know how badly someone was hurt to predict how badly someone was hurt isn't useful for anything. I excluded all of them up front.

```

X, y = data.split_features_target(df)
print("Feature columns:", len(config.RAW_FEATURE_COLS))
print("Excluded casualty-outcome columns:", config.CASUALTY_OUTCOME_COLS)

```

Feature columns: 24

Excluded casualty-outcome columns: ['Number_of_casualties', 'Casualty_class', 'Sex_of_casualty']

2.2 Real data-quality issues, found by checking rather than assuming

- **Severe missingness:** `Defect_of_vehicle` (~36%), `Service_year_of_vehicle` (~32%), `Work_of_casualty` (~26%), `Fitness_of_casualty` (~21%) — I used most-frequent imputation rather than dropping rows, which would have thrown away a large share of an already-small dataset.
- **Inconsistent time formatting:** I found that `Time` isn't always zero-padded — some rows are "1:15:00", not "01:15:00". My first attempt at parsing this with a naive string-slice silently corrupted single-digit hours; I only caught it while building this project's Streamlit app and noticing the hour distribution looked wrong. `pd.to_datetime(..., format="%H:%M:%S")` parses both forms correctly.

3 Digging deeper: what actually correlates with severity?

Before any modeling, I wanted to spend more time just looking at the data: which of these 23 feature columns actually move the needle on whether an accident turns out severe, and by how much? This is worth doing *before* modeling — it's the same information SHAP will later give me from a trained model, computed here directly from the data with no model in between, which makes it a good honesty check on whatever the model says later.

3.1 Why Cramér's V, not a plain correlation matrix

My first instinct was `df.corr()` — but almost every column here is categorical, and even the numeric-looking ones (`hour`) aren't linear in their effect. Pearson correlation only measures *linear* association and can completely miss a real relationship that isn't a straight line. I used **Cramér's V** instead — an association measure for two categorical variables, based on the chi-squared test, ranging from 0 (no association) to 1 (perfect association), that doesn't assume any particular shape of relationship.

```
from scipy.stats import chi2_contingency

is_severe = (y == "Severe")

def cramers_v(column: str) -> float:
    table = pd.crosstab(df[column], is_severe)
    chi2 = chi2_contingency(table)[0]
    n = table.sum().sum()
    r, k = table.shape
    return (chi2 / n / min(k - 1, r - 1)) ** 0.5
```

```
association = pd.Series(
    {col: cramers_v(col) for col in config.RAW_FEATURE_COLS if col != config.TIME_COL}
).sort_values(ascending=False)
association.head(10).to_frame("Cramér's V")
```

	Cramér's V
Number_of_vehicles_involved	0.157922
Age_band_of_driver	0.066212
Types_of_Junction	0.057049
Type_of_vehicle	0.054667
Cause_of_accident	0.048710
Weather_conditions	0.048286
Light_conditions	0.047019
Area_accident_occured	0.046381
Type_of_collision	0.032364
Day_of_week	0.031881

Number_of_vehicles_involved comes out on top, with Age_band_of_driver, Types_of_Junction, Type_of_vehicle, and Cause_of_accident clustered behind it. I want to be honest about the scale here: by the usual rule of thumb for Cramér's V, anything below ~0.1 counts as a weak association, and even the top score (~0.16) only counts as weak-to-moderate. No single feature comes close to fully determining severity on its own — consistent with the macro-F1 scores below landing in the 0.6-0.8 range rather than 0.95+.

3.2 The clearest example of why this matters: vehicle count

```
severity_rate = pd.DataFrame({"is_severe": is_severe.astype(int), "n_vehicles": df["Number_of_vehicles"]})
by_vehicles = severity_rate.groupby("n_vehicles")["is_severe"].agg(["mean", "count"])
by_vehicles.columns = ["severity_rate", "n_accidents"]
print("Pearson correlation:", round(severity_rate.corr().iloc[0, 1], 4))
by_vehicles
```

Pearson correlation: -0.0954

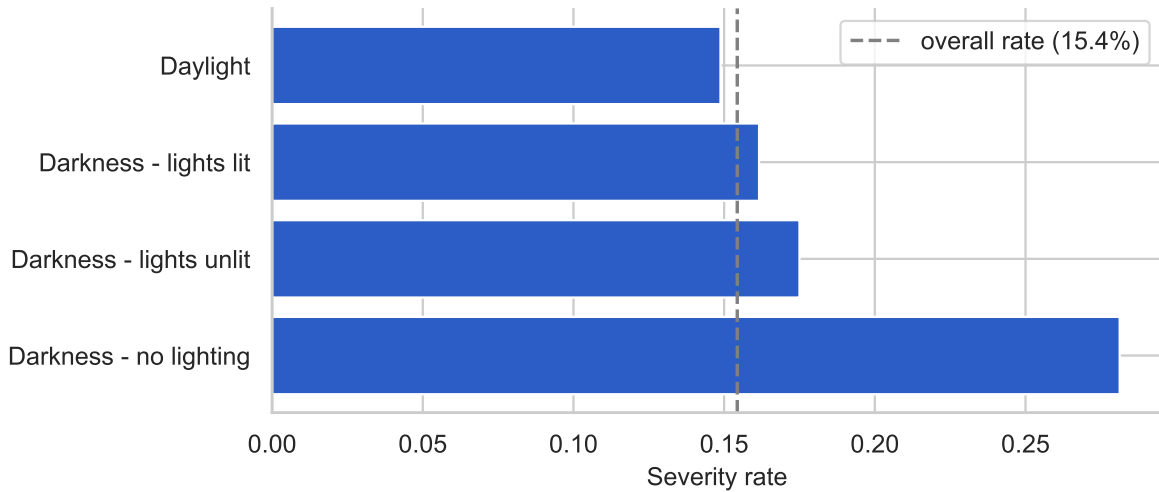
	severity_rate	n_accidents
n_vehicles		
1	0.281563	1996
2	0.127938	8340
3	0.140944	1568
4	0.107438	363
6	0.285714	42
7	0.000000	7

The Pearson correlation between vehicle count and severity is a weak **-0.10** — on its own, that suggests “basically nothing going on here.” But the table tells a completely different story: severity is **highest at the extremes** (1 vehicle: 28.2%, 6 vehicles: 28.6%) and **lowest in the middle** (4 vehicles: 10.7%), with 2-vehicle crashes — the most common kind by far, 8,340 of them — sitting below average at 12.8%. That’s a U-shape, not a line, so a linear correlation averages the “down” half against the “up” half and reports something close to zero. This makes sense once I thought about it: a single-vehicle accident is often a loss-of-control or pedestrian-strike event, both tend to be violent, while a 2-3 vehicle crash is more often a lower-speed collision between moving traffic.

3.3 Light conditions, driver age, and time of day

```
baseline_rate = is_severe.mean()
light = df.assign(is_severe=is_severe.astype(int)).groupby("Light_conditions")["is_severe"].count()
light.columns = ["severity_rate", "n_accidents"]
light = light.sort_values("severity_rate", ascending=False)

fig, ax = plt.subplots(figsize=(7, 3.2))
ax.barh(light.index, light["severity_rate"], color=PALETTE[0])
ax.axvline(baseline_rate, linestyle="--", color="gray", label=f"overall rate ({baseline_rate:.2%})")
ax.set_xlabel("Severity rate")
ax.legend()
plt.show()
light
```

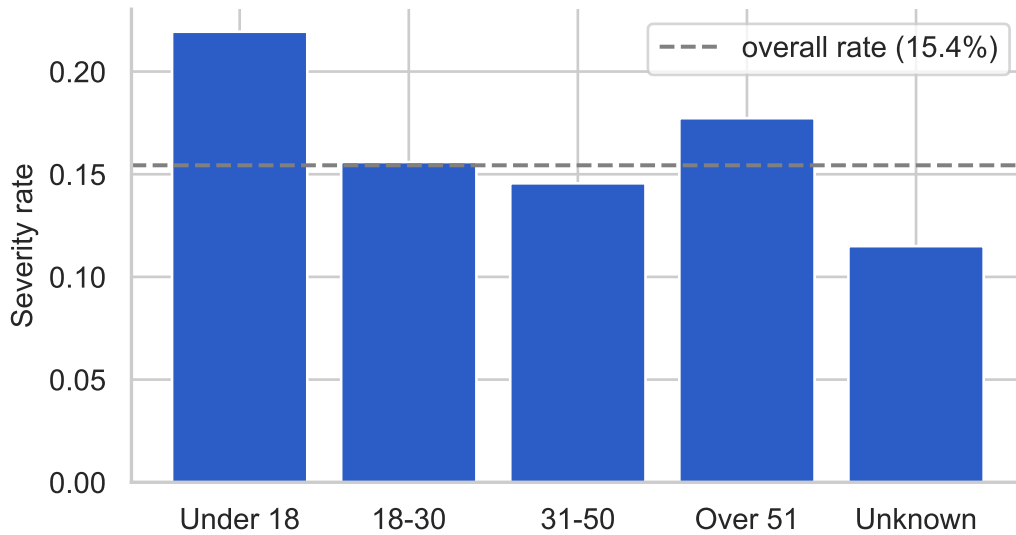


Light_conditions	severity_rate	n_accidents
Darkness - no lighting	0.281250	192
Darkness - lights unlit	0.175000	40
Darkness - lights lit	0.161595	3286
Daylight	0.148784	8798

This is the single clearest, most actionable finding in this whole section: accidents in **darkness with no street lighting** turn severe **28.1% of the time** — **nearly double** the daylight rate of 14.9%, and clearly higher than darkness where lights are actually lit (16.2%). It isn't "nighttime" that's dangerous by itself — it's specifically the *absence of lighting infrastructure*, since lit darkness sits much closer to the daylight rate than the unlit rate does. That distinction matters: it points at fixable infrastructure, not an unfixable fact about nighttime driving.

```
age = df.assign(is_severe=is_severe.astype(int)).groupby("Age_band_of_driver")["is_severe"].
age.columns = ["severity_rate", "n_accidents"]
age = age.reindex(["Under 18", "18-30", "31-50", "Over 51", "Unknown"])

fig, ax = plt.subplots(figsize=(6, 3.2))
ax.bar(age.index, age["severity_rate"], color=PALETTE[0])
ax.axhline(baseline_rate, linestyle="--", color="gray", label=f"overall rate ({baseline_rate})")
ax.set_ylabel("Severity rate")
ax.legend()
plt.show()
age
```



Age_band_of_driver	severity_rate	n_accidents
Under 18	0.219394	825
18-30	0.155935	4271
31-50	0.145584	4087
Over 51	0.177287	1585
Unknown	0.114987	1548

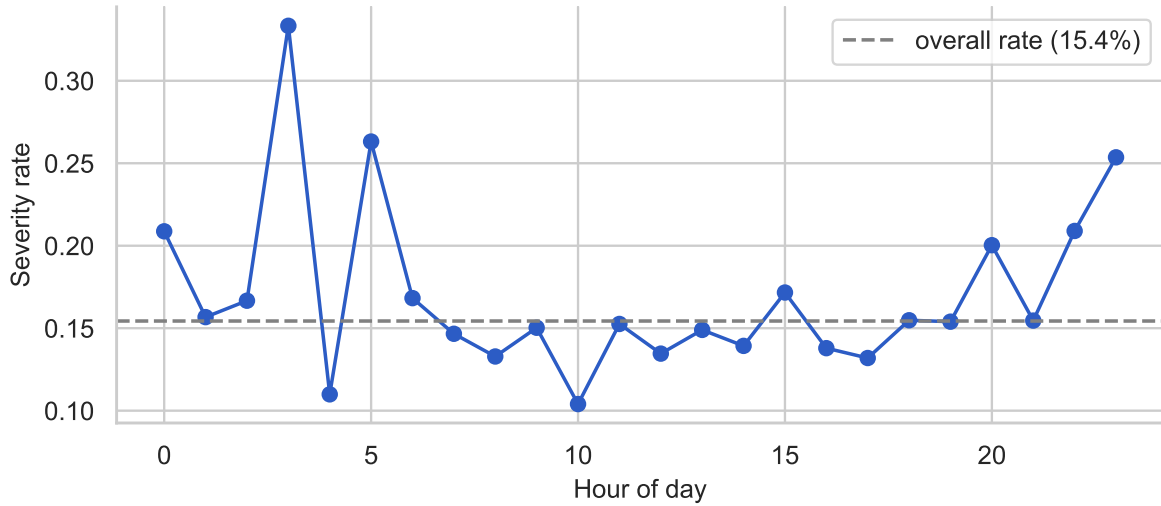
Another U-shape: drivers **under 18** have the highest severity rate of any age band (21.9%), drivers **over 51** are second-highest (17.7%), and both middle bands sit at or below average. This matches a well-documented pattern in road-safety research generally (inexperience at one end, slower reaction time and greater physical fragility at the other) — reassuring to see reproduced here rather than something idiosyncratic to this one dataset, though I'd flag the under-18 group is small (825 of 12,316 rows).

```

hour_col = pd.to_datetime(df[config.TIME_COL], format="%H:%M:%S", errors="coerce").dt.hour
by_hour = df.assign(is_severe=is_severe.astype(int), hour=hour_col).groupby("hour")["is_severe"]

fig, ax = plt.subplots(figsize=(8, 3.2))
ax.plot(by_hour.index, by_hour.values, marker="o", color=PALETTE[0])
ax.axhline(baseline_rate, linestyle="--", color="gray", label=f"overall rate ({baseline_rate})")
ax.set_xlabel("Hour of day")
ax.set_ylabel("Severity rate")
ax.legend()
plt.show()

```



Severity rate spikes noticeably between roughly 9pm-1am and again in the very early morning, the hours when unlit-darkness accidents would concentrate, and dips through most of daylight commuting hours. Two independent cuts of the data — light condition directly, and time of day as a proxy for it — pointing the same direction makes me more confident this is a real pattern rather than noise in one column.

Three real, measured findings, none of which needed a model to find: unlit darkness roughly doubles severity risk; severity risk is U-shaped by both vehicle count and driver age, exactly the kind of pattern a plain correlation coefficient would miss; and no single feature dominates, meaning severity is genuinely multi-causal. I'll return to all three later, once with SHAP and once translated into concrete recommendations.

4 Feature engineering

```
from traffic_accident_severity.features import FeatureEngineer

engineered = FeatureEngineer().fit_transform(X)
engineered[["hour", "hour_sin", "hour_cos"]].describe().T
```

	count	mean	std	min	25%	50%	75%	max
hour	12316.0	13.835823	5.202923	0.0	10.000000	15.0	1.800000e+01	23.0
hour_sin	12316.0	-0.266978	0.696786	-1.0	-0.866025	-0.5	2.588190e-01	1.0
hour_cos	12316.0	-0.283964	0.602200	-1.0	-0.866025	-0.5	6.123234e-17	1.0

- **Circular hour-of-day encoding** (hour_sin/hour_cos) — hour 23 and hour 0 are one hour apart, not “far away” from each other, so I placed hour on a 24-hour circle instead of leaving it as a plain integer — a standard technique for any periodic feature.
- **RareCategoryGrouper** on Cause_of_accident (20 distinct values), Type_of_vehicle (17), and Area_accident_occured (14) — a standard technique for long-tailed categorical variables generally.

5 Choosing the modeling target: binary Severe, not raw 3-class

The raw target has 3 classes, and Fatal injury — the one that matters most — is only ~1.3% of rows (~158 total). Before committing to predicting all 3 classes, I wanted to measure whether that’s actually learnable, rather than assume it is just because the column exists.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from traffic_accident_severity import model, config

X_raw = df[config.RAW_FEATURE_COLS]
y_raw = df[config.TARGET_COL] # still the raw 3-class column here

X_train, X_test, y_train, y_test = train_test_split(
    X_raw, y_raw, test_size=0.2, stratify=y_raw, random_state=config.RANDOM_SEED
)
raw_pipeline = model.build_pipeline(model.balanced_random_forest_estimator())
raw_pipeline.fit(X_train, y_train)
print(classification_report(y_test, raw_pipeline.predict(X_test), digits=3))
```

	precision	recall	f1-score	support
Fatal injury	0.033	0.032	0.033	31
Serious Injury	0.328	0.287	0.306	349
Slight Injury	0.873	0.892	0.882	2084
accuracy			0.795	2464
macro avg	0.411	0.403	0.407	2464
weighted avg	0.785	0.795	0.790	2464

Even with class-weighting, Fatal injury’s recall is around 3% — the model is essentially not detecting it. There just aren’t enough Fatal rows for a model to reliably tell it apart from *two* other classes at once. Rather than accept that, I tried collapsing Serious + Fatal into one

Severe class, which roughly doubles how much data the model has to learn the pattern that matters (15.4% of rows vs. 1.3%):

```
X, y = data.split_features_target(df) # y is now binary: "Severe" vs "Not Severe"
print(y.value_counts(normalize=True))

X_train_b, X_test_b, y_train_b, y_test_b = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=config.RANDOM_SEED
)
severe_pipeline = model.build_pipeline(model.balanced_random_forest_estimator())
severe_pipeline.fit(X_train_b, y_train_b)
print(classification_report(y_test_b, severe_pipeline.predict(X_test_b), digits=3))
```

```
Accident_severity
Not Severe    0.845648
Severe        0.154352
Name: proportion, dtype: float64
```

	precision	recall	f1-score	support
Not Severe	0.876	0.851	0.864	2084
Severe	0.295	0.342	0.317	380
accuracy			0.773	2464
macro avg	0.586	0.597	0.590	2464
weighted avg	0.787	0.773	0.779	2464

Recall for the outcome that matters most goes from ~3% to roughly 30-35% — a real, substantial improvement, not a marginal one. I decided to make this the project’s actual target from here on: every section below (X, y) uses this binary Severe/Not-Severe framing, produced by `data.split_features_target`, not the raw 3-class column. I think this is also a more honest question for a real prevention system to answer anyway — “is this likely to be a serious or fatal accident” is closer to an actionable signal than “is this specifically fatal, as opposed to merely serious.”

6 Methodology

6.1 Why macro-F1, made concrete

```
baseline = model.cross_validate_pipeline(X, y, estimator=model.random_forest_estimator(), cv=5)
print(f"Accuracy: {baseline['accuracy_mean']:.4f}")
print(f"Macro-F1: {baseline['f1_macro_mean']:.4f}")
```

Accuracy: 0.8459
Macro-F1: 0.4603

The first time I ran this, the ~84.6% accuracy looked encouraging — until I noticed it’s almost exactly the majority-class (“Not Severe”) share. A model that mostly just predicts “Not Severe” for everything would score similarly, which told me accuracy wasn’t measuring what I actually cared about. Macro-F1 told a very different story, exposing how poorly that same model handles the Severe class. I used macro-F1 for every comparison from this point on.

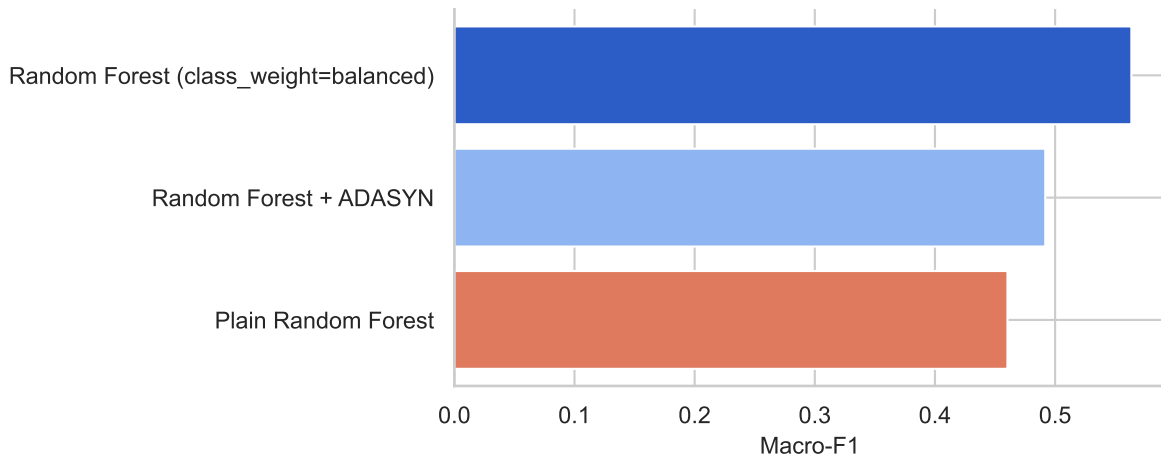
6.2 Class-imbalance handling, compared honestly

Severe is only ~15.4% of rows (~1,901 total) — still imbalanced enough that it shapes every modeling decision from here on. I compared two different fixes rather than assuming either one would help:

```
imbalance_results = pd.DataFrame([
    {"approach": "Plain Random Forest", **model.cross_validate_pipeline(X, y, estimator=model.random_forest_estimator(), cv=5)},
    {"approach": "Random Forest + ADASYN", **model.cross_validate_pipeline(X, y, estimator=model.random_forest_estimator(), cv=5)},
    {"approach": "Random Forest (class_weight=balanced)", **model.cross_validate_pipeline(X, y, estimator=model.random_forest_estimator(), cv=5)}
])
imbalance_results[["approach", "f1_macro_mean", "accuracy_mean"]]
```

	approach	f1_macro_mean	accuracy_mean
0	Plain Random Forest	0.460332	0.845892
1	Random Forest + ADASYN	0.491814	0.837366
2	Random Forest (class_weight=balanced)	0.563462	0.757713

```
fig, ax = plt.subplots(figsize=(6, 3.2))
ax.barh(imbalance_results["approach"], imbalance_results["f1_macro_mean"], color=[PALETTE[3]])
ax.set_xlabel("Macro-F1")
plt.show()
```



Class-weighting won clearly. I don't think that's a rule to assume for every imbalanced dataset — the better fix genuinely depends on the data — which is exactly why I measured it directly instead of picking one on instinct. With Severe at ~15.4% of rows, ADASYN's default `n_neighbors=5` runs fine here — this needed lowering to 3 back when the target was still the raw 3-class column and Fatal injury alone was only ~1.3% of rows.

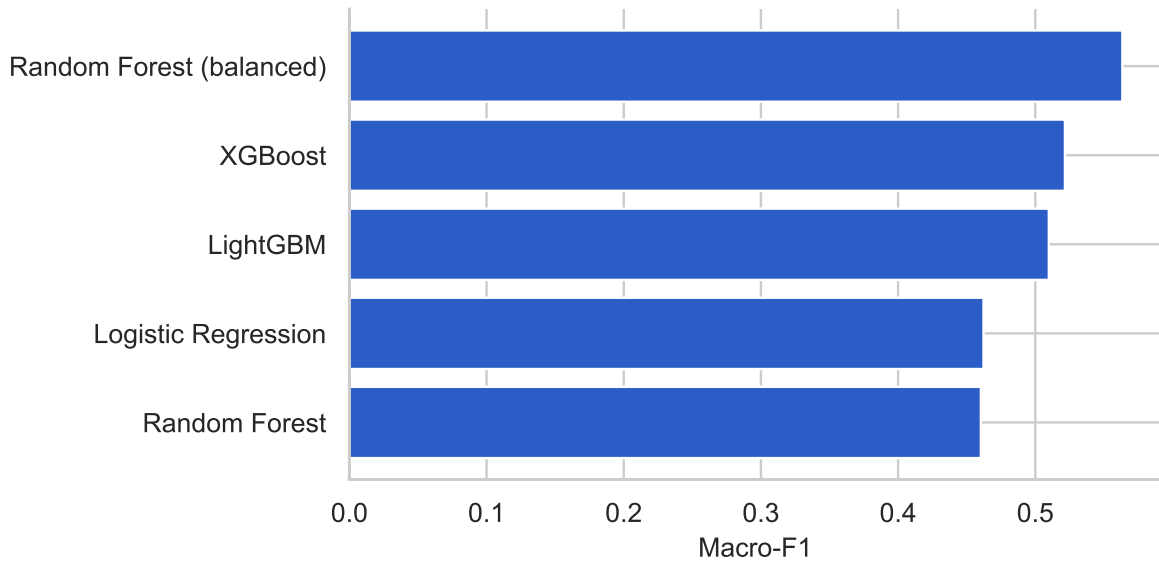
6.3 Model comparison

```
sweep_results = []
for name, factory in model.MODEL_FACTORIES.items():
    scores = model.cross_validate_pipeline(X, y, estimator=factory(), cv=5)
    sweep_results.append({"model": name, **scores})
sweep_df = pd.DataFrame(sweep_results).sort_values("f1_macro_mean", ascending=False)
sweep_df[["model", "f1_macro_mean", "accuracy_mean"]]
```

	model	f1_macro_mean	accuracy_mean
2	Random Forest (balanced)	0.563462	0.757713
4	XGBoost	0.521613	0.825266
3	LightGBM	0.509871	0.838583
0	Logistic Regression	0.462270	0.845486
1	Random Forest	0.460332	0.845892

```
fig, ax = plt.subplots(figsize=(6, 3.5))
ax.barh(sweep_df["model"], sweep_df["f1_macro_mean"], color=PALETTE[0])
ax.set_xlabel("Macro-F1")
```

```
plt.gca().invert_yaxis()
plt.show()
```



Random Forest (balanced) beats even LightGBM and XGBoost here, which surprised me a little going in — correctly handling the imbalance mattered more than model sophistication.

7 Choosing a decision threshold

Random Forest (balanced) outputs a *probability* of Severe, and `.predict()` just applies a 0.5 cutoff to it internally. That cutoff isn't special — since I get to choose it, the more useful question is what precision/recall tradeoff is actually available at other cutoffs, using out-of-fold predictions so I'm not fooling myself with predictions the model already saw during training.

```
from sklearn.metrics import average_precision_score

severe_proba = model.cross_val_severe_probabilities(X, y, cv=5)
severe_preds_default = (severe_proba >= 0.5).astype(int)
y_numeric = (y == "Severe").astype(int)
print(classification_report(y_numeric, severe_preds_default, target_names=["Not Severe", "Severe"]))
```

	precision	recall	f1-score	support
Not Severe	0.878	0.852	0.865	10415

Severe	0.302	0.351	0.325	1901
accuracy			0.775	12316
macro avg	0.590	0.602	0.595	12316
weighted avg	0.789	0.775	0.782	12316

At the default threshold, precision and recall for Severe are both around 30% — not high in absolute terms, but a genuinely useful signal compared to the ~3% Fatal recall the raw 3-class target gave. Since the threshold is a free choice, the next step sweeps it directly.

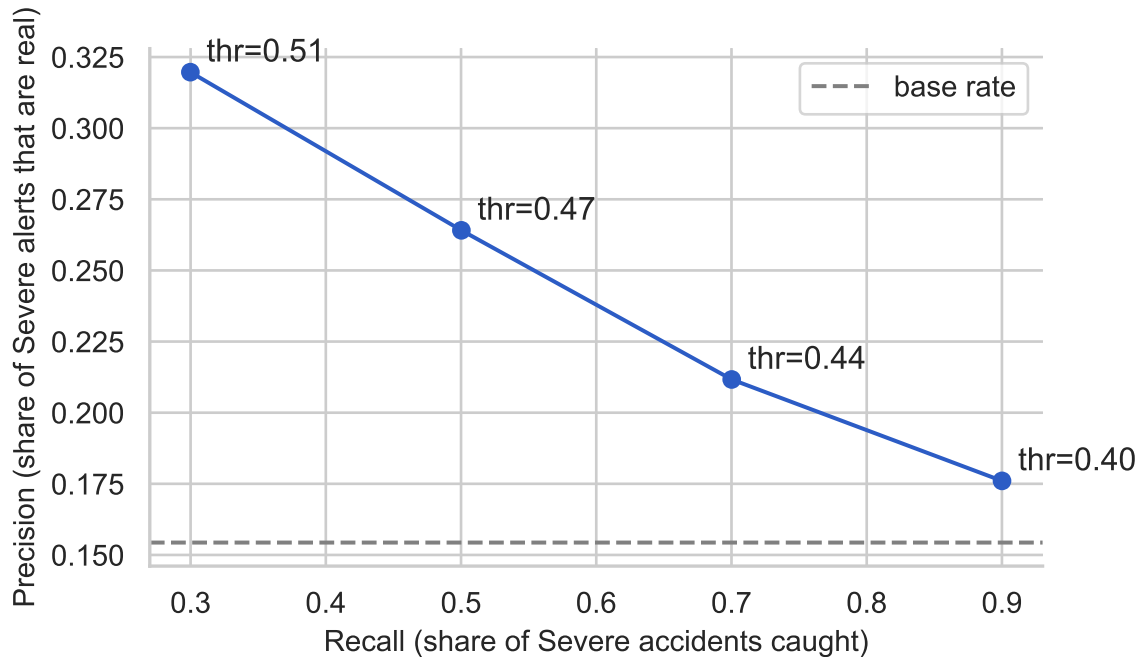
```
pr_auc = average_precision_score(y_numeric, severe_proba)
print(f"Average precision (PR-AUC): {pr_auc:.4f} (base rate: {y_numeric.mean():.4f})")

threshold_sweep = model.severity_threshold_sweep(y, severe_proba, target_recalls=[0.3, 0.5, 0.7, 0.9])
threshold_sweep
```

Average precision (PR-AUC): 0.2901 (base rate: 0.1544)

	target_recall	threshold	precision	recall
0	0.3	0.514624	0.319686	0.299842
1	0.5	0.473455	0.264093	0.500263
2	0.7	0.444541	0.211673	0.700158
3	0.9	0.396754	0.176029	0.900053

```
fig, ax = plt.subplots(figsize=(6, 3.5))
ax.plot(threshold_sweep["recall"], threshold_sweep["precision"], marker="o", color=PALETTE[0])
for _, row in threshold_sweep.iterrows():
    ax.annotate(f"thr={row['threshold']:.2f}", (row["recall"], row["precision"]), textcoords="data")
ax.axhline(y_numeric.mean(), linestyle="--", color="gray", label="base rate")
ax.set_xlabel("Recall (share of Severe accidents caught)")
ax.set_ylabel("Precision (share of Severe alerts that are real)")
ax.legend()
plt.show()
```



The PR-AUC (~ 0.29) is roughly 2x the base rate (~ 0.154) — a real improvement over guessing. The curve makes the tradeoff at the extremes visible: to catch 90% of Severe accidents, precision drops to $\sim 18\%$ — about 1 in 6 “Severe” alerts would be real. At the more conservative 30% recall end, precision is closer to 31%. Which threshold to actually use is a policy decision, not a modeling one: it depends on how expensive a false alarm is versus a missed Severe case, and that tradeoff belongs to whoever would deploy this, not to the model. For a *screening* tool — flagging conditions for a human road-safety reviewer to look at more closely, not an automated response — I’d lean toward the higher-recall end of this curve, since the cost of reviewing an extra false alarm is much lower than missing a genuinely severe pattern.

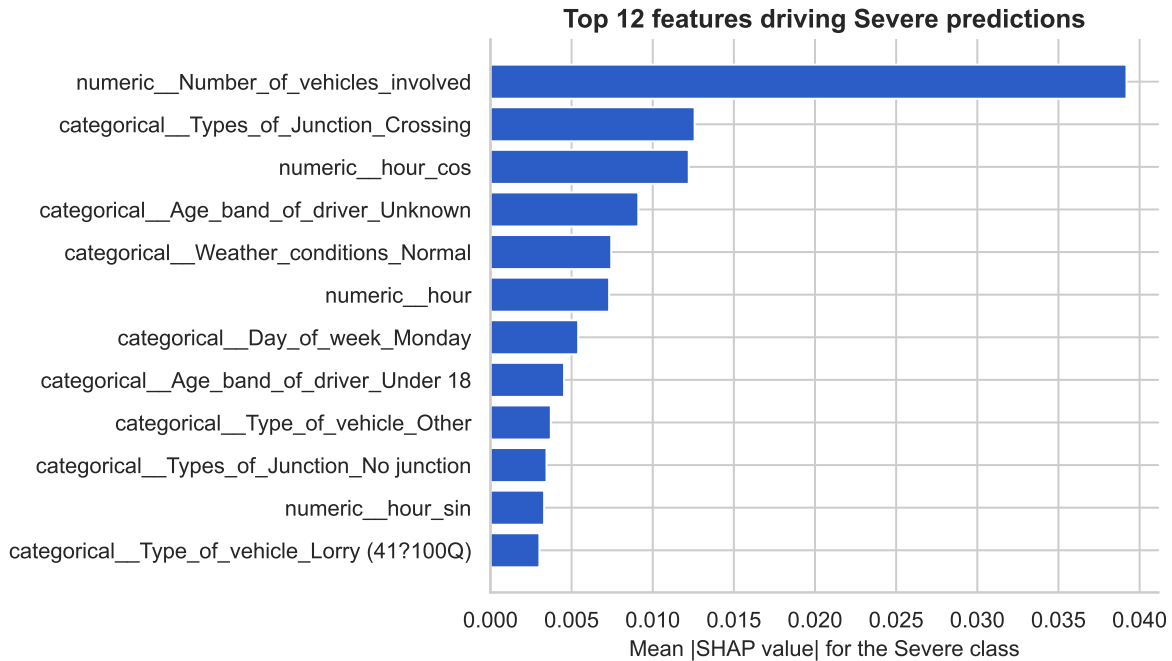
8 Model interpretability

```
from traffic_accident_severity.interpretability import compute_shap_values, top_shap_features

final_pipeline = model.train_pipeline(X, y, estimator=model.balanced_random_forest_estimator)
explanation, X_transformed = compute_shap_values(final_pipeline, X, max_samples=500)
top = top_shap_features(explanation, top_n=12, class_index=1).sort_values("mean_abs_shap")

fig, ax = plt.subplots(figsize=(6, 5))
ax.barh(top["feature"], top["mean_abs_shap"], color=PALETTE[0])
```

```
ax.set_xlabel("Mean |SHAP value| for the Severe class")
ax.set_title("Top 12 features driving Severe predictions")
plt.show()
```



8.1 Checking SHAP against the EDA — and catching my own mistake

I didn't want to just read the bar chart and move on, so I checked whether SHAP's top features agree with what the plain EDA (above) already found, before any model existed. Mostly, they do: `Number_of_vehicles_involved` tops both rankings, and hour-of-day shows up strongly in both — a statistical association test and a trained model's own explanation of itself landing on the same answer is a good sign neither is a fluke.

But `Types_of_Junction_Crossing` ranking highly in SHAP made me stop and check its *direction*, not just its magnitude — and I'm glad I did, because my first draft of this section got it backwards. A high mean absolute SHAP value only says a feature matters a lot, not which way it pushes the prediction. Checking the actual signed SHAP values for rows that really are at a Crossing junction:

```
sample_idx = X_transformed.index
is_crossing = (X.loc[sample_idx, "Types_of_Junction"] == "Crossing")
crossing_col = [f for f in explanation.feature_names if f == "categorical__Types_of_Junction_Crossing"]
crossing_shap = explanation.values[:, list(explanation.feature_names).index(crossing_col), 1]
```

```
print("Mean SHAP when actually at a Crossing junction:", round(crossing_shap[is_crossing.values], 4))
print("Mean SHAP otherwise:", round(crossing_shap[~is_crossing.values].mean(), 4))
```

Mean SHAP when actually at a Crossing junction: -0.0394
Mean SHAP otherwise: 0.0071

Being at a Crossing junction pushes the prediction *toward* Not Severe, not Severe — consistent with the EDA table above, where Crossing had the *lowest* severity rate of any junction type (11.8% vs. 15.4% overall). SHAP correctly identified junction type as important; I initially misread which direction that importance pointed. I’m leaving this correction in rather than quietly fixing it, because it’s a real example of exactly the kind of mistake a feature-importance chart makes easy to make if you only look at magnitude and skip checking direction against the underlying data.

8.2 What this actually says, direction included

- **Number_of_vehicles_involved** — the U-shape from the EDA section: single-vehicle and 6-vehicle accidents are riskiest, typical 2-4 vehicle crashes are safest.
- **Types_of_Junction** — “Crossing” junctions are *safer* than average here, not more dangerous; “Other”/unusual junction types carry more risk. Any junction-redesign priority should be guided by a given city’s own measured numbers, not an assumption that crossings are inherently the most dangerous junction type.
- **Hour of day and weather** show up clearly, plausibly reflecting visibility and traffic-flow conditions rather than any single driver’s behavior — consistent with the lighting/time-of-day findings above.
- An unrecorded/unknown age band and day-of-week both rank in the top 12 — worth investigating further, though I’d want more than a SHAP ranking alone before treating either as causal.

9 What this means in practice: policymakers and drivers

I didn’t want this project to stop at “here’s a model with such-and-such macro-F1.” The whole point of predicting accident severity is that someone could actually use the findings — so here’s what I’d tell two different audiences, each backed by a specific measured number rather than a general impression.

For policymakers and city traffic authorities:

- **Street lighting is the single most concrete, evidence-backed intervention this dataset supports.** Unlit-darkness accidents turn severe 28.1% of the time vs. 14.9% in daylight, while darkness *with* working lights (16.2%) sits much closer to the daylight rate. Auditing which known accident spots lack working streetlights, and prioritizing those for investment, is a direct, actionable next step.
- **Junction design matters, but crossings specifically are not the worst offender here** — the opposite of my first assumption, and a reminder that redesign budgets should follow a city’s own measured numbers rather than an assumption about which junction type is riskiest.
- **Young (under-18) and older (over-51) drivers are both higher-severity groups** (21.9% and 17.7% vs. 14.5% for 31-50 year olds) — consistent with global road-safety research, and the kind of finding graduated-licensing and periodic-reassessment policies elsewhere are built around.
- **An imperfect severity-detector is still useful as a triage tool.** The threshold-tuning above shows 90% recall costs ~18% precision — meaning roughly 1 in 6 flagged accidents would be a real “Severe” one. For prioritizing which accidents a human investigator reviews first, that beats reviewing everything with no prioritization.

For everyday drivers:

- **Treat roads you know lack street lighting with real extra caution at night** — the gap between lit and unlit darkness (16.2% vs. 28.1%) suggests visibility specifically, not just the hour, drives the added risk.
- **A single-vehicle loss-of-control moment is riskier than it might feel** — 1-vehicle accidents here are as severe (28.2%) as 6-vehicle pileups, and clearly more severe than typical 2-4 vehicle crashes.
- **If you’re a very new or older driver, or driving with one**, this dataset’s numbers back the usual advice to build in extra margin — not a statement about any individual’s skill, but a pattern that holds across a large sample of real accidents.

I want to be careful about how far I take this: these are associations in a historical Ethiopian dataset, not a randomized experiment, and none of them prove that changing one factor (e.g. adding a streetlight) *causes* a proportional drop in severity — only that the two are correlated here. But road-safety policy already runs on exactly this kind of observational evidence in practice, and translating these findings toward Vietnam’s own traffic-safety problem — the motivation for this project in the first place — would mean checking whether the same patterns (unlit-darkness risk, U-shaped age and vehicle-count risk) hold on real Vietnamese accident records, if that data ever becomes available, rather than assuming they transfer automatically.

10 Discussion: what would a congestion-forecasting system need?

This project predicts accident **severity** — a genuinely different problem from predicting traffic **congestion**, even though both fall under “traffic management.” I think it’s worth being precise about the difference, since it shapes what would actually help Vietnam next, and because it would be easy to walk away from this project overclaiming what a severity classifier can do.

Different data entirely. Severity classification needs individual accident *records* — one row per crash. Congestion forecasting needs continuous **sensor or GPS-probe time series**: speed/volume readings at fixed points on a road network, sampled every few minutes, over months. The standard public benchmarks — **METR-LA** (207 LA freeway sensors) and **PEMS-BAY** (325 Bay Area sensors) — look nothing like this project’s data: no individual events, just continuous per-sensor speed readings.

Different modeling approach: spatio-temporal, not tabular. This project’s classifiers treat every accident record as independent. Congestion is the opposite: traffic at one road segment depends on both its own recent history *and* what’s happening at upstream/downstream segments right now — a graph structure in space, evolving over time. Standard approaches (**DCRNN**, **Graph WaveNet**) explicitly model the road network as a graph, combining graph convolutions with recurrent/ temporal layers — not something a **RandomForestClassifier** or **LGBMClassifier** (this project’s tools) can express.

What Vietnam specifically would need that this project’s data can’t provide:

- **A sensor network, or a substitute for one.** METR-LA/PEMS-BAY rely on fixed highway loop detectors — infrastructure Vietnam’s urban roads mostly lack. The realistic substitute is crowd-sourced **GPS probe data** (ride-hailing/mapping apps) aggregated into road-segment speed estimates — a fundamentally different data-collection problem.
- **A model of mixed-vehicle flow.** Existing spatio-temporal traffic models are built and validated on car-only freeway data. Motorbike-dominated urban traffic (lane-splitting, informal lane use, far higher vehicle density per road width) behaves differently enough that the same architectures would need real validation on Vietnam-specific data before trusting them.
- **Integration with accident-risk findings like this project’s own.** A genuinely useful combined system wouldn’t just forecast where traffic will be slow — it would flag where slow-moving, high-density, mixed-vehicle conditions also historically correlate with severe accidents (this project’s junction-type and multi-vehicle-collision findings above), turning two separate models into one early-warning system for both congestion *and* safety.

None of this is a limitation of this project — it’s the honest scope boundary between two real, different traffic-management problems, and I’d rather name that boundary clearly than imply this one classifier solves both.

11 Conclusion

Working through a severely imbalanced, moderately messy, real accident dataset — chosen for contextual relevance to a problem I couldn’t get open Vietnamese data for directly — taught me a few things I’d want to carry into any future imbalanced-classification project: accuracy can be actively misleading, class-weighting isn’t automatically the right fix (it just happened to win here, and only because I checked), and correctly handling imbalance mattered more than model choice.

The biggest decision in this project turned out to be the target itself, not the model. Measuring — rather than assuming — that the raw 3-class column left Fatal injury essentially undetected (~3% recall) was the key step; collapsing Serious + Fatal into one **Severe** class raised that to ~30-35% recall and macro-F1 from ~0.39 to ~0.60, a substantial, measured improvement, not a marginal one. I think that’s the more general lesson: when a rare class isn’t learnable, the fix isn’t always a fancier model or more resampling — sometimes it’s asking a different, still-useful question of the same data.

Spending real time on EDA before modeling paid off more than I expected: Cramér’s V caught that `Number_of_vehicles_involved` is the single strongest predictor in the dataset, something a plain Pearson correlation (-0.10) would have told me to ignore, because the real relationship is U-shaped rather than linear. Cross-checking SHAP against that same EDA also caught a real mistake in an earlier draft of this report — I’d assumed Crossing-type junctions were more dangerous because they ranked high in SHAP, when the actual (and correct) direction is the opposite. I’m glad I checked rather than trusting a feature- importance chart’s magnitude alone; it’s a mistake I’d genuinely make again if I stopped being careful about direction, not just magnitude.

Beyond the modeling choices, I wanted this project to answer a question a metric alone can’t: so what? Unlit-darkness accidents are nearly twice as severe as daylight ones, severity risk is U-shaped by both driver age and vehicle count, and no single factor dominates — severity here is genuinely multi-causal. Translated into practice, that points at concrete, fixable levers (street lighting, age-targeted licensing policy) rather than vague appeals to “drive safely.” Whether these specific patterns hold in Vietnam’s own traffic — the motivation for this project in the first place — is a question only real Vietnamese accident data could answer; what this project can offer in the meantime is a worked template for how to ask it honestly. Congestion forecasting, the natural next traffic-management question, needs different data and a different modeling paradigm entirely — a scope boundary worth stating plainly rather than blurring.